

LaTeX 从入门到数学公式与 Beamer

VS Code + LaTeX Workshop 可编译教程

2026 年 7 月 22 日

摘要

本教程结合四份 LaTeX 课程讲义，使用一个真实、可编译的多文件项目，演示中文文档、文字排版、图表、数学公式、定理、参考文献和 Beamer。建议在 VS Code 左侧阅读源码，在右侧对照生成的 PDF。

目录

1 入门：从源文件到 PDF	4
1.1 为什么使用 LaTeX	4
1.2 TeX、LaTeX、XeLaTeX、Biber 与 LaTeXmk	4
1.3 第一个最小文档	4
1.4 在 VS Code 中编译和预览	5
1.5 本章检查清单	6
2 文档结构：命令、环境与页面组成	7
2.1 命令、参数与环境	7
2.2 文档类和文档选项	7
2.3 导言区与宏包	8
2.4 标题、摘要与目录	8
2.5 章节层次与自动编号	9
2.6 页面设置	9
3 文字排版：段落、字体与列表	10
3.1 段落、换行与分页	10
3.2 粗体、斜体、下划线与颜色	10

3.3	字号与作用范围	10
3.4	特殊字符	11
3.5	无序、编号与描述列表	11
3.6	原样显示代码	12
3.7	空格和间距的正确用法	12
4	表格与图片：浮动体和交叉引用	14
4.1	tabular 的行、列与对齐	14
4.2	booktabs 三线表	14
4.3	插入和缩放图片	14
4.4	浮动体、标题与标签	15
4.5	交叉引用	15
5	数学基础：从行内公式到分段函数	16
5.1	文字模式与数学模式	16
5.2	行内、行间与编号公式	16
5.3	上下标、分式和根式	17
5.4	求和、积分、极限和函数	17
5.5	希腊字母与数学字体	17
5.6	定界符、数组和分段函数	17
5.7	数学公式中的文字与间距	18
6	数学进阶：对齐、定理与参考文献	19
6.1	align 多行公式	19
6.2	split 与长公式	19
6.3	矩阵	20
6.4	子公式	20
6.5	定理、定义与证明	20
6.6	公式和页码交叉引用	21
6.7	BibLaTeX 与 Biber	21
7	大型项目：拆分、命令与排错	22
7.1	为什么拆分文件	22

7.2	input 和主文件	23
7.3	自定义命令	23
7.4	相对路径	23
7.5	常见辅助文件	24
7.6	固定排错顺序	24
7.7	清理后重新构建	25
8	Beamer: 制作演示文稿	26
8.1	Beamer 是什么	26
8.2	frame 是基本页面	26
8.3	标题页和目录页	27
8.4	block、exampleblock 与 alertblock	28
8.5	主题、导航和画面比例	28
8.6	逐项显示	28
8.7	fragile frame 与代码	29
8.8	阅读 beamer-demo.tex	29

1 入门：从源文件到 PDF

学习目标 / Learning goals

学完本章后，你将理解 TeX、LaTeX、XeLaTeX、Biber 和 LaTeXmk 这五种工具的关系，并能在 VS Code 中编译一个最小中文文档。

1.1 为什么使用 LaTeX

LaTeX 把“写什么”和“长什么样”分开：你在纯文本源文件中标记标题、段落、公式、图片和引用，再由编译工具统一生成 PDF。这种工作方式特别适合结构较长、公式较多、需要交叉引用或多人使用同一版式的文档。源文件可以逐行阅读、搜索和比较，版式规则则集中写在主文件的导言区，因而修改一处就能保持全文一致。

1.2 TeX、LaTeX、XeLaTeX、Biber 与 LaTeXmk

工具	作用	本项目中的位置
TeX	最底层的排版系统，提供命令展开、断行和分页等基础能力。	其他工具共同依赖的排版基础。
LaTeX	建立在 TeX 之上的文档系统，提供文档类、章节、列表和交叉引用。	<code>main.tex</code> 与各章节所使用的标记语言。
XeLaTeX	能直接处理 Unicode 和系统字体的 LaTeX 编译引擎。	负责把本项目的中文源文件编译成 PDF。
Biber	读取 <code>.bib</code> 数据库并生成参考文献所需信息。	处理根目录中的 <code>references.bib</code> 。
LaTeXmk	根据依赖关系自动重复运行 XeLaTeX，并在需要时调用 Biber。	LaTeX Workshop 构建配方调用的自动化工具。

1.3 第一个最小文档

把下面内容保存为一个 `.tex` 文件，并使用 XeLaTeX 编译：

```
\documentclass[UTF8]{ctexart}

\begin{document}
你好，LaTeX!
\end{document}
```

编译效果 / Build result

生成的 PDF 中只会出现“你好，LaTeX!”。`\documentclass` 以及 `\begin{document}` 之前的导言区用于配置文档，它们本身不会被打印到 PDF。

1.4 在 VS Code 中编译和预览

本项目已经启用自动保存 (auto save) 和保存后自动构建 (build on save)。停止输入约 1 秒后，VS Code 会自动保存当前源文件；这次保存会触发默认的第一条配方 `latexmk` (XeLaTeX)，由 `LaTeXmk` 调用 `XeLaTeX` 更新 PDF。若想立即手工构建，仍可按 `Command+Option+B`。

1. 打开项目文件夹 (open the project folder)。
2. 打开 `main.tex` (open `main.tex`)。
3. 修改一小处，停止输入约 1 秒，等待自动保存和默认 `XeLaTeX` 配方完成构建。
4. 按 `Command+Option+V`，在 VS Code 右侧打开 PDF 预览。
5. 把光标放在一行源码中，按 `Command+Option+J`，从源码定位到 PDF 对应位置。
6. 在 PDF 中双击一段文字，返回左侧对应的源码位置。

按 `Command+Option+J` 的“源码到 PDF”和双击 PDF 的“PDF 回到源码”合称 `SyncTeX` (源码与 PDF 同步定位)。前者也叫正向定位 (forward search)，后者也叫反向定位 (inverse search)；它们让你不必在长源码和长 PDF 中重复滚动查找同一段内容。

注意 / Note

`.tex` 文件是可以用编辑器打开和修改的纯文本；PDF 是编译后生成的输出文件。要修改内容，应编辑 `.tex` 源文件并重新编译，而不是直接修改 PDF。

常见错误 / Common error

下面的命令缺少一个右花括号：

```
\textbf{这行少了右花括号
```

编译器随后可能报告很多连锁错误。请先找到并修复日志中第一条有用的错误信息，然后重新编译；后面的错误往往会随之消失。

1.5 本章检查清单

- ☐ 我能指出 `\documentclass` 用来选择文档类型和选项。
- ☐ 我能写出从 `\begin{document}` 到 `\end{document}` 的文档环境。
- ☐ 我知道本项目的根文件是 `main.tex`。
- ☐ 我知道停止输入约 1 秒会自动保存，并触发默认的 XeLaTeX 配方。
- ☐ 我会使用 `Command+Option+B` 立即手工编译。
- ☐ 我会使用 `Command+Option+V` 在右侧打开 PDF 预览。
- ☐ 我会使用 `Command+Option+J` 从源码定位到 PDF。
- ☐ 我会双击 PDF 返回源码，并知道这两个方向合称 SyncTeX。

2 文档结构：命令、环境与页面组成

学习目标 / Learning goals

学完本章后，你将能区分命令、参数和环境，理解文档类、导言区与宏包的职责，并能组织标题、摘要、目录、章节层次和页面设置。

2.1 命令、参数与环境

LaTeX 命令通常以反斜杠开始。方括号中的可选参数用来微调行为，花括号中的必选参数则告诉命令要处理什么内容。下面是两种通用写法；它们位于源码框中，只作为示意，不会被当前文档执行。

```
\command[可选参数]{必选参数}
```

```
\begin{环境名}
```

环境中的内容

```
\end{环境名}
```

在真实命令 `\textbf{粗体文字}` 中，`textbf` 是命令名，花括号中的文字是必选参数；实际执行后得到 **粗体文字**。花括号负责划定参数范围，本身不会显示在 PDF 中。

环境像一间有明确入口和出口的房间：`\begin` 打开房间，`\end` 关闭房间，中间的内容按照该环境的规则排版。

常见错误 / Common error

下面的列表用 `itemize` 开始，却错误地用 `enumerate` 结束：

```
\begin{itemize}
  \item 第一项
\end{enumerate}
```

每个环境的开始名称和结束名称必须完全相同；这里应把 `\end{enumerate}` 改成 `\end{itemize}`，否则编译器会报告环境不匹配。

2.2 文档类和文档选项

`\documentclass` 通常是主文件中第一条有效命令，而且一篇文档只选择一个文档类。这里的 `ctexart` 是适合中文短文和教程的文章类；`UTF8`、`12pt` 和 `a4paper` 分别指定编码、基础字号和纸张大小。

```
\documentclass[UTF8,12pt,a4paper]{ctexart}
\usepackage[left=2.6cm,right=2.6cm]{geometry}
```

类选项写在文档类名之前的方括号中；若省略某个可选项，文档类会使用对应的默认值。不同文档类支持的选项可能不同，因此不要把某个类的专用选项机械复制到另一个类中。

2.3 导言区与宏包

从 `\documentclass` 之后到 `\begin{document}` 之前的区域叫作导言区。它适合加载宏包、定义颜色和命令、设置页边距，以及填写标题信息；导言区的配置通常不会作为普通正文直接打印出来，却会影响整篇文档。

`\usepackage` 用来加载宏包。例如，`geometry` 管理页面尺寸和页边距，本教程还在主文件中加载了数学、图片、表格、列表、颜色和超链接等宏包。把共享配置放在 `main.tex` 中，可以避免每个章节重复设置。

注意 / Note

导言区用于“设置怎样排版”，`document` 环境用于“写要排版的内容”。普通段落、列表和章节标题应放在 `\begin{document}` 之后。

2.4 标题、摘要与目录

标题、作者和日期通常在导言区声明；进入 `document` 环境后，`\maketitle` 才把这些信息排版出来。摘要由 `abstract` 环境包围，`\tableofcontents` 根据带编号的章节命令生成目录。

```
\title{文档标题}
\author{作者姓名}
\date{\today}

\begin{document}
\maketitle
\begin{abstract}
这里是摘要。
\end{abstract}
\tableofcontents
\end{document}
```


`\today` 会在编译时生成当天日期。目录依赖辅助文件记录章节与页码，所以刚加入或移动标题后，通常需要再次编译；本项目的 LaTeXmk 会自动完成所需轮次。

2.5 章节层次与自动编号

LaTeX 使用 `\section`、`\subsection` 和 `\subsubsection` 表示从高到低的章节层次。本小节就是一个实际执行的 `subsection`：LaTeX 自动给它编号，并把它加入目录，而无需手工填写数字或页码。

无编号的小标题示例

这行标题由实际的 `\subsubsection*` 命令生成。带星号的标题 **没有编号，也不会自动加入目录**。如果确实需要让无编号标题出现在目录中，应再使用专门的目录命令，而不是手写一个看似正确的页码。

编译效果 / Build result

使用章节命令后，新增、删除或移动小节时，LaTeX 会重新计算编号、目录和页码。因此标题文字中不应手写“2.1”之类的序号。

2.6 页面设置

纸张大小可以由文档类选项指定，页边距则常交给 `geometry` 宏包统一管理。本教程在主文件中使用 A4 纸，并分别设置上下左右页边距；这样各章节共享同一版心，不需要在章节文件里重复调用宏包。

需要从新页开始时，可以使用 `\clearpage`。它不仅换页，还会先安排尚未排出的浮动图片和表格。`\pagestyle{plain}` 则使用带页码的简洁页眉页脚样式。

本章检查 / Chapter check

我能解释命令参数与环境边界，指出导言区和正文的分界，按正确顺序组织标题、摘要与目录，并说明普通标题与星号标题在编号和目录行为上的差别。

3 文字排版：段落、字体与列表

学习目标 / Learning goals

学完本章后，你将能用空行划分自然段，控制文字样式与字号，正确输入特殊字符，选择合适的列表环境，并使用语义清楚的间距命令。

3.1 段落、换行与分页

在 LaTeX 源码中，一个空行会结束当前自然段并开始下一段；同一自然段中的普通换行通常只被当作一个空格。下面的源码保留了一个空行，因此会产生两个自然段，而最后两行仍属于同一段。

这是第一个自然段。

这是第二个自然段。

LaTeX 会自动换行。

需要在同一自然段内明确换行时，可以使用 `\\`；只想建议一个较合适的分页位置时，可以使用 `\pagebreak[2]`。`\newpage` 会立即开始新页，`\clearpage` 还会先排出等待中的浮动图片和表格。分页命令应按内容结构使用，不要靠连续空行把文字推到下一页。

编译效果 / Build result

第一行在这里结束。

第二行由 `\\` 主动开始，但两行仍属于同一个自然段。

3.2 粗体、斜体、下划线与颜色

文字样式命令只影响花括号中的内容，因此可以在一个句子里安全地组合普通文字和强调文字：

粗体文字，*斜体文字*，带下划线的文字，蓝色强调文字。

粗体适合强调关键词，斜体常用于术语或轻度强调。下划线和颜色应少量使用，以免喧宾夺主；颜色名称 `TutorialBlue` 已在 `main.tex` 中统一定义。

3.3 字号与作用范围

花括号可以建立局部作用范围。下面三个字号各自在自己的分组中生效，分组结束后正文恢复正常字号：

小号文字 正常文字 较大文字

如果漏掉分组，像 `\Large` 这样的声明式命令会继续影响后面的文字。教程正文通常应保持统一基础字号，只在标题、图注或确有层次需要的位置调整大小。

3.4 特殊字符

有些字符在 LaTeX 源码中负责命令、注释、参数或对齐，不能直接当作普通正文输入。下表左列用 `\verb` 原样显示源字符，右列执行转义命令并给出真正的排版结果。

源字符	可见输出
#	#
\$	\$
%	%
&	&
-	—
{	{
}	}
\	\

常见错误 / Common error

原始的 `%` 会把同一源码行中它后面的内容变成注释。例如下面一行本来想输出完整句子，但“今天有效”不会进入 PDF：

```
折扣是 20% 今天有效
```

要输出百分号，应写成 `\%`，例如“折扣是 20% 今天有效”。

3.5 无序、编号与描述列表

无序列表适合没有先后关系的并列内容：

- 段落由空行划分。
- 字体命令用花括号限定范围。
- 特殊字符需要正确转义。

编号列表适合必须按顺序完成的步骤：

1. 保存 `.tex` 源文件。

2. 使用 XeLaTeX 编译文档。
3. 打开 PDF 检查结果。

描述列表适合解释一组术语：

源码 写在 `.tex` 文件中的文字与命令。

引擎 读取源码并完成排版的程序，例如 XeLaTeX。

输出 编译后得到的 PDF 及辅助文件。

嵌套列表可以表达从属关系，但这里保持最多两层，避免层级过深：

- 正文元素
 - 段落与列表
 - 字体与间距
- 页面元素

3.6 原样显示代码

本教程的 `codeblock` 环境由 `fvextra` 提供。环境内的反斜杠、花括号和空格都按源码原样显示，所以适合教学示例；其中的命令是惰性的，不会改变当前文档。

```
\textbf{这行只显示源码，不会变成粗体。}  
\begin{itemize}  
  \item 这也不会生成真正的列表。  
\end{itemize}
```

短小的行内源码可以用 `\verb`，较长或包含多行的源码则应使用 `codeblock`。

3.7 空格和间距的正确用法

LaTeX 会把源码中的连续普通空格合并。需要不可断行的词组时使用波浪号，例如 第~3~章；需要明确的水平间距时，可以选择与含义相符的命令。

编译效果 / Build result

紧邻文字；小间距：甲 乙；中等间距：甲 乙；指定间距：甲 乙。

注意 / Note

重复输入多个空格不能可靠地对齐文字，因为 LaTeX 会合并普通空格，而且不同内容的宽度也不同。需要成行成列时，应使用表格或对齐环境；只需要局部间距时，再使用 `\,、\quad` 或 `\hspace{...}` 等专用命令。

本章检查 / Chapter check

我能用空行开始新段落，控制样式与字号的作用范围，正确输出特殊字符，选择三类基本列表，在代码框中安全展示源码，并用表格、对齐环境或专用命令处理间距。

4 表格与图片：浮动体和交叉引用

学习目标 / Learning goals

学完本章后，你将能读写基本表格、制作三线表、按正文宽度缩放图片，并用浮动体、标题、标签和交叉引用保持编号自动更新。

4.1 tabular 的行、列与对齐

`tabular` 环境的列格式写在花括号中：`l` 表示左对齐，`c` 表示居中，`r` 表示右对齐。每一行用 `&` 分隔相邻单元格，再用 `\\` 结束当前行。比如列格式 `{lcr}` 会依次建立左对齐、居中和右对齐的三列；源码中每行也必须提供与这三列相匹配的内容。

4.2 booktabs 三线表

`booktabs` 宏包提供 `\toprule`、`\midrule` 和 `\bottomrule`。下面的活动表格保留清晰的三线结构，并由 LaTeX 自动编号。

表 1: 常用文档类型比较		
文档类型	适合内容	是否有 chapter
article / ctexart	短文、报告	否
book / ctexbook	书籍、长文档	是
beamer	幻灯片	不适用

4.3 插入和缩放图片

`graphicx` 宏包提供 `\includegraphics`。使用 `width=0.82\textwidth`，可以让图片宽度保持为当前正文宽度的 82%，比写死厘米数更容易适应不同页面设置。

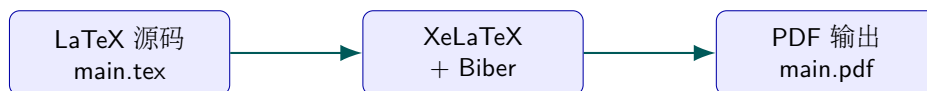


图 1: 从 LaTeX 源码到 PDF 的构建流程

常见错误 / Common error

下面的命令指向一个不存在的文件；它只作为惰性源码显示，不会在本教程中执行：

```
\includegraphics{figures/missing-file.pdf}
```

相对路径应从主文档的编译位置理解。本项目从 `main.tex` 所在目录编译，所以 `figures/example-figure.pdf` 表示该目录下 `figures` 文件夹中的图片。若文件名、扩展名、大小写或相对层级写错，LaTeX 就无法找到图片。

4.4 浮动体、标题与标签

`figure` 和 `table` 都是浮动体。位置参数中的 `h` 建议放在当前位置附近，`t` 建议放在页顶，`b` 建议放在页底，`p` 建议放到浮动体专页；它们都是候选位置，不是把对象固定在某一点的绝对命令。LaTeX 会根据对象大小、页面余量和浮动规则选择实际位置。

`\caption` 生成图题或表题以及对应编号，`\label` 保存这个编号。通常把标签紧跟在标题之后，并用带类型前缀的名称区分图片和表格，例如 `fig:build-flow` 与 `tab:document-types`。

4.5 交叉引用

使用 `\ref` 可以读取标签所记录的编号。例如，表 1 比较了三类常用文档，图 1 展示了从 `main.tex` 到 `main.pdf` 的构建流程。加入或移动带标签的对象后需要再次编译，让辅助文件先写入编号、再把编号读回正文；本项目的 LaTeXmk 会自动完成这些轮次。

本章检查 / Chapter check

我能解释 `l`、`c`、`r`、`&` 和 `\` 的含义，能区分浮动位置建议与固定位置，并能用 `\label` 和 `\ref` 自动维护图表编号。

5 数学基础：从行内公式到分段函数

学习目标 / Learning goals

学完本章后，你将能区分文字模式与数学模式，选择行内、行间或编号公式，并正确输入上下标、分式、根式、运算符、希腊字母、定界符、数组和分段函数。

5.1 文字模式与数学模式

普通中文和英文写在文字模式中；变量、运算符和数学关系应放进数学模式。例如，下面句子中的公式使用一对美元符号进入和退出行内数学模式：

质能方程 $E = mc^2$ 位于正文之中。

数学模式会把字母默认排成数学变量的斜体，并按照公式规则解释上标、下标和空格。因此源码中的 `^`、`_` 和 `\alpha` 等写法不能随意放在普通正文里。

5.2 行内、行间与编号公式

短公式适合放在正文中；需要突出或结构较大的公式适合单独成行。推荐使用 LaTeX 的 `\[...\]` 形式书写无编号行间公式：

$$e^{i\pi} + 1 = 0$$

需要编号和交叉引用时，使用 `equation` 环境，并把 `\label` 放在环境内部：

$$F = ma \tag{1}$$

公式 (1) 是牛顿第二定律。引用标签而不是手写编号，以后增删公式时编号会自动更新。

常见错误 / Common error

下面的旧式写法只作为惰性源码展示，不会执行：

```
$$
  e^{i\pi}+1=0
$$
```

它不属于推荐的 LaTeX 文档写法，间距和错误检查也不如标准环境一致。无编号行间公式请使用 `\[...\]`，需要编号时请使用 `equation`。

5.3 上下标、分式和根式

上标用 `^`，下标用 `_`；当内容不止一个字符时，要用花括号分组。

$$x_i^2, \quad \frac{a+b}{c}, \quad \sqrt[n]{x}, \quad \sum_{i=1}^n i, \quad \int_0^1 x^2 dx$$

`\frac{分子}{分母}` 生成分式，`\sqrt[n]{x}` 生成 n 次根式。求和与积分的上下限也分别由下标和上标给出。

5.4 求和、积分、极限和函数

LaTeX 为常用函数提供了 `\sin`、`\cos`、`\ln` 和 `\lim` 等命令。它们会把函数名排成直立字体，并自动提供合适间距；若直接输入斜体字母，读者可能把它误解为若干变量的乘积。

$$\sin^2 \theta + \cos^2 \theta = 1, \quad \ln e = 1, \quad \lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$$

积分中的微分符号写成 `\mathrm{d}` 很有用，因为这里的 d 是直立的运算记号，不是变量 d ；前面的 `\`，则加入一小段易读的间距。

5.5 希腊字母与数学字体

希腊字母使用命令输入，例如小写 α 、 β 和大写 Γ 。`\mathbb` 常用于数集，`\mathbf` 可把拉丁字母排成粗体，`\bm` 则也能可靠地加粗希腊字母等数学符号。

$$\alpha + \beta \in \mathbb{R}, \quad \mathbf{x} = (x_1, x_2), \quad \boldsymbol{\alpha} = (\alpha_1, \alpha_2), \quad \Gamma(n) = (n-1)!$$

5.6 定界符、数组和分段函数

`\left` 与 `\right` 会让成对定界符随内部内容自动伸缩。`array` 环境使用与表格相似的列格式和对齐符号，适合在数学模式中排列元素：

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

`cases` 环境会生成左花括号并按条件对齐，特别适合定义分段函数：

$$\left(\frac{a}{b}\right), \quad |x| = \begin{cases} x, & x \geq 0, \\ -x, & x < 0. \end{cases}$$

5.7 数学公式中的文字与间距

数学模式会忽略源码中的普通空格。需要短文字说明时使用 `\text{...}`，需要精细间距时使用 `\,`、`\quad` 等命令：

$$f(x) = 0 \quad \text{当且仅当} \quad x = 0, \quad \int_a^b f(x) \, dx$$

这里的 `\text{当且仅当}` 保留了正常文字字体和中文间距，而积分号后的 `\,`，把被积函数与微分符号清楚地区分开。

本章检查 / Chapter check

我能根据上下文选择行内、行间或编号公式，用语义化命令输入函数、微分符号和数学字体，并能写出自动伸缩定界符、数学数组以及可编译的 `cases` 分段函数。

6 数学进阶：对齐、定理与参考文献

学习目标 / Learning goals

学完本章后，你将能用 `align`、`split`、矩阵和子公式组织复杂数学内容，使用定义、定理与证明环境，并通过标签和 BibLaTeX 维护交叉引用与参考文献。

6.1 `align` 多行公式

`align` 环境用 `&` 标记对齐位置，常把它放在等号之前；每行末尾的 `\\` 开始下一行。下面两行各自带有标签，之后可以独立引用。

$$(a + b)^2 = a^2 + 2ab + b^2, \tag{2}$$

$$(a - b)^2 = a^2 - 2ab + b^2. \tag{3}$$

例如，公式 (2) 给出和的平方，公式 (3) 给出差的平方。需要取消某一行的编号时，可以在该行末尾使用 `\notag`。

注意 / Note

下面是只用于辨认旧源码的惰性示例：

```
\begin{eqnarray}
  a + b & = & b + a \\
  ab & = & ba
\end{eqnarray}
```

这种写法会在关系符周围产生不理想的间距，也不便于细致控制编号。新文档应使用 `align` 环境，并只放置一个用于对齐的 `&`。

6.2 `split` 与长公式

当整个推导只需要一个编号时，可以把 `split` 放进 `equation`：

$$\begin{aligned} (x + y + z)^2 &= (x + y)^2 + 2(x + y)z + z^2 \\ &= x^2 + y^2 + z^2 + 2xy + 2xz + 2yz. \end{aligned} \tag{4}$$

这里的两行共享公式 (4) 的一个编号。外层 `equation` 管理编号，内层 `split` 管理

换行和对齐。

6.3 矩阵

`pmatrix` 适合圆括号矩阵；类似地，`bmatrix` 会生成方括号。矩阵内部的排列方式和表格相似，但它必须位于数学模式中。

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \det(A) = -2.$$

这里的 `\det` 会把行列式函数名排成直立字体，`\qquad` 则在矩阵和计算结果之间加入易读的水平间距。

6.4 子公式

一组密切相关的公式可以放进 `subequations`。外层标签记录整组编号，内部的 `align` 负责逐行排列：

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2, \tag{5a}$$

$$v(t) = v_0 + a t. \tag{5b}$$

式组 (5) 把匀加速运动的位置与速度公式归在同一个主编号下。若还要分别引用子公式，可在各行继续添加不同的 `\label`。

6.5 定理、定义与证明

导言区已经用 `amsthm` 定义了本教程的 `definition` 和 `theorem` 环境；`proof` 则由宏包直接提供。

定义 6.1. 若一个大于 1 的自然数，其正因数只有 1 和它自身，则称它为素数。

定理 6.2 (勾股定理). 直角三角形两条直角边满足 $a^2 + b^2 = c^2$ 。

证明. 这里用一个简短文字说明证明环境的排版形式。

□

方括号中的文字为定理补充名称；证明环境结束时会自动排出证毕符号。

6.6 公式和页码交叉引用

公式 (2) 位于第 19 页，而式组 (5) 由一个标签代表整组公式。`\eqref{...}` 会自动加上公式编号所需的圆括号，`\pageref{...}` 则适合在长文档中提示目标所在页。标签名中的 `eq:` 前缀也能帮助读者一眼识别引用对象的类型。

6.7 BibLaTeX 与 Biber

Knuth 对文学编程的系统论述和 Lamport 的 LaTeX 教材都是理解排版工作流的经典资料 [1, 2]。正文中的 `\cite{...}` 只写稳定的文献键；作者、标题、年份等完整数据保存在 `references.bib` 中。`biblatex` 负责引用样式，`Biber` 读取数据库并解析引用，最后由主文档中的 `\printbibliography` 输出正文实际引用的条目。

本章检查 / Chapter check

我能选择 `align`、`split`、矩阵或子公式组织数学内容，能用定理与证明环境表达数学论述，并能通过 `\label`、`\eqref`、`\pageref` 和 `\cite` 维护自动更新的引用。

7 大型项目：拆分、命令与排错

学习目标 / Learning goals

学完本章后，你将能把长文档拆成多个章节，判断 `\input` 和相对路径的起点，定义不会意外覆盖已有命令的新命令，辨认常见辅助文件，并按固定顺序定位编译问题。

7.1 为什么拆分文件

文档变长后，把所有内容放进一个 `main.tex` 会让搜索、审阅和多人协作越来越困难。更稳妥的做法是让根文件集中保存文档类、宏包、全局样式、章节顺序和参考文献输出，再让每个章节文件只负责一个清楚的主题。这样可以单独定位内容，又不会复制全局配置。

下面是本教程完成后的项目树。它位于惰性源码环境中，只展示目录关系，不会让 LaTeX 尝试载入尚未讲解的文件。

```
latex-tutorial/  
|-- .vscode/  
|   |-- settings.json  
|-- figures/  
|   |-- example-figure.pdf  
|   |-- example-figure.tex  
|-- sections/  
|   |-- 01-getting-started.tex  
|   |-- 02-document-structure.tex  
|   |-- 03-text-formatting.tex  
|   |-- 04-tables-and-figures.tex  
|   |-- 05-mathematics.tex  
|   |-- 06-theorems-and-references.tex  
|   |-- 07-large-projects.tex  
|   |-- 08-beamer.tex  
|-- main.tex  
|-- main.pdf  
|-- beamer-demo.tex  
|-- beamer-demo.pdf  
|-- references.bib  
|-- README.md
```

7.2 input 和主文件

根文件按阅读顺序载入章节；`\input` 会把目标文件的内容插入当前位置，通常可以省略 `.tex` 扩展名。例如，`main.tex` 中的前两章写作：

```
\input{sections/01-getting-started}
\input{sections/02-document-structure}
```

章节文件不是独立文档：它们不重复 `\documentclass`、宏包或 `document` 环境。即使当前正在编辑 `sections/07-large-projects.tex`，也应把 `main.tex` 作为根文件编译。文件开头的编辑器提示 `% !TEX root = ../main.tex` 能帮助 LaTeX Workshop 从子文件回到正确入口；真正的构建结果仍由根文件中的章节顺序和全局配置决定。

7.3 自定义命令

重复出现的语义和样式可以封装成命令。`[1]` 表示命令接收一个参数，定义中的 `#1` 会被调用时花括号里的内容替换。下面两行是活动内容，会实际参与本教程编译：

这里突出显示一个 **重要术语**。

命令必须在第一次使用之前定义；全篇共用的命令通常放在根文件导言区。LaTeX 还会检查新命令的名称是否已经存在。如果新名称与已有命令重名，它会报告命令已定义，而不会静默覆盖原行为。只有确实要修改已知命令时，才使用 `\renewcommand`，并且应先确认原命令的参数形式。

7.4 相对路径

本项目从 `main.tex` 所在的 `latex-tutorial/` 目录构建。根文件中的路径都从这个目录开始解释，例如：

- `sections/07-large-projects` 指向章节源文件；
- `figures/example-figure.pdf` 指向图片；
- `references.bib` 指向文献数据库。

不要因为 `\input` 命令写在某个子文件中，就自动把路径起点改成该子文件所在目录。

注意 / Note

许多构建环境区分文件名和目录名的大小写，源码拼写必须与磁盘完全一致。若从子文件启动了错误的编译入口，正确路径也会失效；此时先确认根文件，再检查路径。

7.5 常见辅助文件

编译会在源文件旁生成若干中间文件。它们记录跨轮次信息，通常不应手工编辑：

<code>.aux</code>	保存标签、引用和其他需要下一轮读取的交叉引用信息。
<code>.log</code>	保存引擎的完整构建记录，是定位第一条有效错误的主要依据。
<code>.toc</code>	保存目录条目，下一轮编译时由 <code>\tableofcontents</code> 读取。
<code>.out</code>	常由 <code>hyperref</code> 写入书签和链接相关信息。
<code>.bcf</code>	把引用和排序等控制信息交给 Biber。
<code>.bbl</code>	保存 Biber 根据 <code>.bib</code> 数据生成、供 LaTeX 读取的文献内容。
<code>.blg</code>	保存 Biber 的处理日志和书目诊断信息。
<code>.synctex.gz</code>	保存源码行与 PDF 位置的映射，支持正向和反向跳转。
<code>.pdf</code>	是最终排版输出；修改内容后应由源码重新生成，而不是直接编辑它。

源文件 `.tex`、文献库 `.bib` 和图片资源是项目输入，不能当作辅助文件删除。辅助文件虽然可以重建，但在正常增量构建中能帮助 LaTeXmk 判断依赖并减少不必要的工作。

7.6 固定排错顺序

编译器可能在一次失败后输出许多连锁信息。先处理最早一条与自己源码有关的红色错误，再重新构建，往往比从日志末尾倒查更有效。常见现象可以按下表缩小范围：

现象	首先检查	常见原因
Undefined control sequence	命令拼写与宏包	命令拼错、命令尚未定义或缺少提供它的宏包。
File not found	路径与大小写	根文件不对、相对层级错误或资源没有放进项目。
Missing } inserted	相关行之前的花括号	命令参数少了右花括号，错误位置可能晚于真正缺失处。
Environment ended	begin/end 名称	环境名不一致、嵌套顺序错误或漏写结束命令。
undefined references	标签名与构建轮次	标签拼错，或修改编号后还没有完成足够的编译轮次。
empty bibliography	引用键、数据源与 Biber	正文没有有效引用、键不匹配、文献库未载入或 Biber 未运行。

无论具体错误名称是什么，都按下面顺序逐项检查，并在每次只改一个原因后重新构建：

1. Read the first useful red error. (阅读第一条有用的红色错误。)
2. Check braces. (检查花括号是否成对。)
3. Check begin/end names. (检查环境开始和结束名称。)
4. Check file paths and capitalization. (检查文件路径与大小写。)
5. Check required packages. (检查所需宏包。)
6. Rebuild references. (重新构建交叉引用和文献引用。)
7. Clean auxiliary files and rebuild. (清理辅助文件后重新构建。)

常见错误 / Common error

不要只看最后一行，也不要一次删除或重写大段源码。例如日志中的 `Undefined control sequence` 只是现象名称；应回到它指向的第一处相关源码，判断是拼写、定义顺序还是宏包问题。修好第一处后重新编译，后续连锁错误可能会一起消失。

7.7 清理后重新构建

交叉引用、目录或文献状态偶尔会因中断构建而过期。先保存所有源文件并正常重新构建；只有问题仍然存在、且日志提示旧的辅助状态时，才在项目根目录执行完整清理：

```
/Library/TeX/texbin/latexmk -C main.tex  
/Library/TeX/texbin/latexmk -synctex=1 -interaction=nonstopmode  
↪ -file-line-error -xelatex main.tex
```

第一条命令会移除可再生成的辅助文件和旧输出，第二条命令从 `main.tex` 重新建立目录、交叉引用、Biber 文献数据、SyncTeX 映射和 PDF。清理不是修复源码错误的替代品：如果命令拼写、花括号、环境名或路径本身有误，干净构建仍会在同一根因处失败。

编译效果 / Build result

章末检查：确认项目只有明确的根文件，各章节由 `\input` 载入；新增命令先定义后使用且名称不冲突；路径从实际构建入口解释；出现问题时先修第一条有效错误，最后才清理辅助文件并完整重建。

8 Beamer：制作演示文稿

学习目标 / Learning goals

学完本章后，你将能创建一个独立的 Beamer 根文件，用 `frame` 组织页面，添加标题、目录、信息块、公式、图片和双栏内容，使用 `\pause` 控制逐项显示，并在含逐字源码的页面上正确使用 `fragile` 选项。

8.1 Beamer 是什么

Beamer 是 LaTeX 的演示文稿文档类。普通文章围绕连续段落和自动分页组织内容，而 Beamer 围绕一张张幻灯片组织内容；每张幻灯片应只表达一个清楚的重点。它仍然使用熟悉的命令、数学环境、图片和交叉引用，因此可以复用前面章节的写作方法。

本项目中的 `beamer-demo.tex` 是独立根文件，不由 `main.tex` 载入。编辑演示文稿时应直接编译它，得到对应的 `beamer-demo.pdf`。

8.2 frame 是基本页面

下面是一个完整、可单独保存和编译的最小示例。它位于源码展示环境中，只供阅读，不会执行其中的命令。

```
\documentclass[aspectratio=169,11pt]{beamer}
\usepackage[UTF8]{ctex}
\usetheme{Boadilla}

\begin{document}
\begin{frame}
  \frametitle{第一张幻灯片}
  这里是内容。
\end{frame}
\end{document}
```

`\documentclass{beamer}` 选择演示文稿文档类；`aspectratio=169` 把画布设为现代宽屏常用的 16 : 9；11pt 是正文基准字号。每个 `frame` 环境产生一个逻辑页面，`\frametitle` 设置该页面的标题。只有放在 `\begin{document}` 与 `\end{document}` 之间的 `frame` 才会输出。

8.3 标题页和目录页

标题、作者和日期通常在导言区定义，再分别用 `\titlepage` 和 `\tableofcontents` 输出。标题页加上 `plain` 选项后会隐藏页眉、页脚，把视觉注意力集中在标题上。

```
\title{我的 Beamer 演示}
\author{演示者姓名}
\date{\today}

\begin{frame}[plain]
  \titlepage
\end{frame}

\begin{frame}{目录}
  \tableofcontents
\end{frame}
```

目录内容来自 `\section` 和 `\subsection`。与文章一样，修改章节后应让 LaTeXmk 完成必要的多轮编译，使目录和导航信息稳定。

`\section{章节标题}` 建立正式章节，并把标题写入目录、页眉页脚等导航信息。如果希望观众在进入新主题前先看到清楚的“章节开始页”，可在导言区注册专用钩子。`AtBeginSection` 钩子会在每次执行 `section` 时先运行指定内容。

```
\AtBeginSection[]{
  \begin{frame}[plain]
    \sectionpage
  \end{frame}
}
```

三条命令的分工如下：

- `AtBeginSection` 决定何时插入。
- `frame` 创建逻辑页面。
- `sectionpage` 显示当前章节标题。

由于每个正式 `section` 都会自动增加一个 `frame`，规划总页数时要把这些章节开始页计算在内；示例中的 `plain` 只改变版式，不会取消计数。

8.4 block、exampleblock 与 alertblock

Beamer 提供三种常用信息块。选择环境时应依据内容语义，而不是只为了颜色：普通说明用 `block`，正向示例用 `exampleblock`，必须注意的风险用 `alertblock`。

```
\begin{block}{普通信息}
这是普通说明。
\end{block}

\begin{exampleblock}{示例}
这是示例内容。
\end{exampleblock}

\begin{alertblock}{提醒}
这是重要提醒。
\end{alertblock}
```

8.5 主题、导航和画面比例

主题控制标题栏、页脚和列表等整体结构，颜色主题只调整配色。独立示例选择 Boadilla 主题和 seahorse 颜色主题，并移除投影时通常不需要的导航图标：

```
\documentclass[aspectratio=169,11pt]{beamer}
\usetheme{Boadilla}
\usecolortheme{seahorse}
\setbeamertemplate{navigationsymbols}{{}}
```

`aspectratio=169` 必须写在文档类选项中，因为它决定整份演示文稿的画布比例。主题命令放在导言区，可以保证所有 frame 使用一致样式。先确定清楚、稳定的结构，再少量调整颜色，通常比逐页手工设置更容易维护。

8.6 逐项显示

`\pause` 会从当前位置建立一个新的覆盖层。演示时，观众先看到第一项，继续翻页后依次看到后续内容：

```
\begin{frame}{逐项显示}
  \begin{itemize}
    \item 先提出问题。
```

```

        \pause
        \item 再展示方法。
        \pause
        \item 最后给出结论。
    \end{itemize}
\end{frame}

```

这个源码只有一个逻辑 frame，却会在 PDF 中形成三个物理页面。覆盖层适合控制讲解节奏，但不应把每个短句都拆开，否则翻页次数会妨碍理解。

8.7 fragile frame 与代码

逐字源码包含大量反斜杠和花括号，LaTeX 必须先把整个 frame 写入临时文件再读取。因此，只要 frame 中使用 verbatim 一类逐字环境，就要在 frame 选项中添加 fragile；遗漏它通常会导致编译错误。

```

\begin{frame}[fragile]{最小源码}
  \begin{verbatim}
    \begin{frame}
      \frametitle{示例页面}
      这里是内容。
    \end{frame}
  \end{verbatim}
\end{frame}

```

fragile 不是对所有页面都必需的装饰选项，只应加在确实含逐字源码的 frame 上。这样既表达了页面的特殊读取需求，也让问题定位更直接。

8.8 阅读 beamer-demo.tex

打开 beamer-demo.tex 后，先读导言区中的文档类、中文支持、主题、导航设置和 \AtBeginSection 钩子。源码手写八个 frame，两个正式 section 又各自自动生成一张章节开始页，所以共有十个逻辑 frame。数学公式与本地图片共用一页，源码/PDF 双栏与总结要点共用一页；第九个逻辑 frame 中的两次 \pause 让它产生三个覆盖层，最终把十个逻辑 frame 扩展为十二个物理 PDF 页面。

编译时在项目根目录运行：

```

/Library/TeX/texbin/latexmk -synctex=1 \
  -interaction=nonstopmode -file-line-error \

```

```
-xelatex beamer-demo.tex
```

编译效果 / Build result

章末检查: 确认演示文稿使用独立根文件和 16 : 9 画布; 每个 frame 只表达一个重点; 主题、颜色和导航在导言区统一设置; 用 `\AtBeginSection` 与 `\sectionpage` 生成章节开始页; `\pause` 只服务于讲解顺序; 含逐字源码的页面使用 `fragile`; 最后直接编译 `beamer-demo.tex` 并检查生成的 PDF。

参考文献

- [1] Donald E. Knuth. “Literate Programming”. In: *The Computer Journal* 27.2 (1984), pp. 97–111. DOI: [10.1093/comjnl/27.2.97](https://doi.org/10.1093/comjnl/27.2.97).
- [2] Leslie Lamport. *LaTeX: A Document Preparation System*. 2nd ed. Addison-Wesley, 1994.